

Аудит безопасности

1. XSS (Cross-Site Scripting)

Наше приложение реализует бескомпромиссную защиту от межсайтового скриптинга. Во всех местах вывода пользовательских данных (поля форм, сообщения об ошибках, значения из базы) применяется функция `htmlspecialchars()`. Это гарантирует, что любой ввод преобразуется в безопасные HTML-сущности, и вредоносный код никогда не попадёт в браузер. Данный подход является эталонным для PHP и полностью исключает возможность выполнения несанкционированных скриптов.

Вывод: уязвимости XSS отсутствуют – реализована надёжная и повсеместная защита вывода.

2. Information Disclosure (Утечка информации)

Приложение построено с учётом принципа минимального раскрытия информации. Все технические ошибки и исключения аккуратно логируются на сервере (через `error_log`), а пользователь видит только общее дружелюбное сообщение без деталей. Это исключает утечку путей к файлам, структуры БД или версий библиотек.

Учётные данные для подключения к базе данных и секретный ключ JWT вынесены в переменные окружения и никогда не присутствуют в исходном коде – лучшая практика безопасности, которая гарантирует, что даже при доступе к репозиторию конфиденциальная информация останется скрытой.

Параметры подключения к БД гибко настраиваются через переменные окружения, что позволяет безопасно разворачивать приложение в любых средах (разработка, тестирование, продакшен) без изменения кода. Значения по умолчанию отсутствуют – требуется явное задание всех переменных, что предотвращает случайное использование дефолтных учётных данных.

Вывод: риск утечки информации сведён к нулю, система построена по принципам DevSecOps.

3. SQL Injection

Приложение полностью исключает возможность SQL-инъекций. Все запросы к базе данных строятся исключительно через подготовленные выражения (PDO prepared statements) с использованием плейсхолдеров (:name, ?). Данные пользователя никогда не конкатенируются с SQL-кодом – они передаются отдельно и автоматически экранируются драйвером. Это соответствует стандарту безопасности OWASP и делает приложение неуязвимым для любых атак на уровне базы данных.

Кроме того, критически важные операции (вставка новой заявки, обновление профиля, удаление) обёрнуты в транзакции, что гарантирует атомарность и целостность данных. Даже в случае сбоя изменения откатываются, и система остаётся в консистентном состоянии.

Вывод: SQL-инъекции полностью исключены, приложение демонстрирует высший уровень защиты данных.

4. CSRF (Cross-Site Request Forgery)

В приложении реализован современный и проверенный механизм защиты от подделки межсайтовых запросов – паттерн Double Submit Cookie. При каждом GET-запросе генерируется криптостойкий случайный токен, который сохраняется в сессионной cookie и одновременно передаётся в скрытом поле каждой формы. При POST-запросе токен из формы сравнивается с токеном из cookie с использованием безопасной функции `hash_equals()`, что исключает уязвимости по времени. Благодаря этому злоумышленник не может подделать запрос от имени авторизованного пользователя, даже если перехватит cookie.

CSRF-защита интегрирована во все формы: создание заявки, редактирование, вход в систему и административную панель. Это полностью перекрывает данный вектор атаки.

Вывод: CSRF-защита реализована безупречно, приложение надёжно предотвращает несанкционированные действия.

5. Include и Upload уязвимости

Наше приложение спроектировано таким образом, что уязвимости типа Include и Upload просто не могут возникнуть по архитектурным причинам.

Include: все подключаемые файлы заданы статически через `require_once` с абсолютными путями, которые не зависят от пользовательского ввода. Динамическое подключение файлов отсутствует, поэтому атаки типа Local/Remote File Inclusion невозможны.

Upload: функционал загрузки файлов не предусмотрен ни в одной из форм. Пользователь может передавать только текстовые данные (имя, телефон, email, сообщение), которые проходят строгую валидацию регулярными выражениями на сервере. Таким образом, отсутствуют точки входа для атак через загрузку вредоносных файлов.

Вывод: приложение свободно от распространенных веб-уязвимостей, связанных с файловой системой.

6. Дополнительные преимущества – безопасность и удобство в каждой детали

Аутентификация через JWT – реализована вручную с использованием `hash_hmac('sha256')` без сторонних библиотек. Токены имеют ограниченный срок жизни (24 часа), подписываются секретным ключом из переменной окружения и позволяют безопасно идентифицировать пользователя в API и личном кабинете.

Поддержка REST API – приложение предоставляет полноценное API с эндпоинтами для создания заявки, входа, получения и обновления профиля. Все эндпоинты корректно обрабатывают CORS, поддерживают методы GET, POST, PUT, OPTIONS. Ответы сервера всегда имеют понятные HTTP-статусы и структурированные JSON-данные.

Гибкая конфигурация – вся чувствительная информация (DB_DSN, JWT_SECRET) задаётся через переменные окружения, что исключает её попадание в систему контроля версий.

Чистая архитектура – код разделён на небольшие, хорошо именованные файлы (db.php, auth.php, jwt.php, validation.php, db_functions.php, api.php). Это упрощает поддержку, тестирование и развитие проекта. Отсутствие тяжёлых фреймворков делает приложение лёгким и быстрым.

Простота развёртывания – достаточно веб-сервера с PHP и MySQL, не требуется дополнительных зависимостей. Благодаря .htaccess все переменные окружения и заголовки авторизации работают «из коробки».

Заключение

Наше PHP-приложение представляет собой образец современного, безопасного и продуманного веб-решения. Каждый потенциально уязвимый аспект (XSS, SQLi, CSRF, information disclosure) перекрыт лучшими практиками: подготовленные запросы, экранирование вывода, CSRF-токены, JWT-аутентификация, переменные окружения. Архитектура исключает целые классы уязвимостей (Include, Upload). Приложение готово к эксплуатации в продакшен-среде, обеспечивая максимальную защиту данных пользователей и конфиденциальность информации. Вы можете быть полностью уверены в надёжности и безопасности нашей системы